

A Complete GAN on Chip: Generator, Discriminator and Piecewise-Linear Activations in Open-PDK 180 nm

William Anthony, I Made Medika Surya W.M., John Ruben Saragi,
Mahardhika Putra Adipratama, and Fairuz Aquilla Rahagi

Bandung Institute of Technology, Bandung, Indonesia

E-mail: {13223048, 13222021, 13222049, 13222081, 13222107}@mahasiswa.itb.ac.id

Abstract—A prior 8-bit accelerator for a GAN generator on the open GF180MCU 180 nm process kept two things off the die: the nonlinear activations, which existed only as an 8193-entry lookup table inside a testbench, and the discriminator, which existed only in Python. This paper closes both gaps and reports the resulting chip. A nine-segment piecewise-linear unit evaluates tanh, sigmoid, ReLU, LeakyReLU and a log mantissa on one shared multiplier, at a cost of 1.71 gray levels out of 255 against the lookup table it replaces, and it is what makes an on-chip generator physically possible. The discriminator is then shown to cost almost no hardware at all: it is the same MAC array and the same buffers under two values of a destination-select register, so a full adversarial forward pass, both binary cross-entropy losses and 28 metric registers fit behind an unchanged pin budget. Right-sizing the operand buffers across three SRAM depths yields 13 macros totalling $990,581 \mu\text{m}^2$, 8.6% less silicon than the 16 uniform macros it replaces, while paying for four-way batching that improves compute throughput $4.26\times$; a parallel burst command on the host link improves weight streaming $23.63\times$, and together they take end-to-end latency from 9.45 s to 0.161 s per image. The macro hardens at 40 ns on a single 3.3 V supply with zero DRC, LVS and antenna violations, reproducible to a byte-identical placement database. We also report a negative-to-positive result on antenna closure that contradicts our own earlier conclusion, and explain what made the first reading wrong.

I. INTRODUCTION

Generative models are usually presented as a data-center workload, but the smallest of them are not large. A three-layer multi-layer-perceptron generator for MNIST is roughly half a megabyte of INT8 weights, and the arithmetic behind one image is a few hundred thousand multiply-accumulates. That is well inside what an open-PDK 180 nm die can hold, and it makes such a model a useful vehicle for asking which parts of a generative pipeline are genuinely hardware and which merely look like it.

Our earlier accelerator [1] answered part of that question: it mapped the generator’s dense layers onto a 4×4 output-stationary array backed by eleven physical SRAM macros and took it through a fully open-source flow to a signed-off chip. Two things it did not answer, and could not, were the ones that make a GAN a GAN. Its tanh was an 8193-entry Q20 lookup table that existed only as an array inside a testbench — convenient in simulation and impossible to tape out at that

size. Its discriminator existed only in the Python reference. The chip could therefore compute the generator’s matrix products, but not the generator, and certainly not the adversarial pair.

This paper reports what happens when both are brought on chip. The contributions are:

- A nine-segment piecewise-linear activation unit that serves all six functions the network needs from one shared multiplier, with measured error against the exact functions and against the lookup table it replaces (Section IV).
- The observation, with its cost accounting, that the discriminator is almost free: it is not a separate datapath but two values of a destination-select register on the existing array (Section III).
- A memory right-sizing across three SRAM depths that is simultaneously smaller and faster than the uniform arrangement it replaces, and the batching it pays for (Section V).
- A host link whose burst commands remove the interface as the dominant latency term, measured at the pads (Section VI).
- A physical implementation with a clean sign-off, and a methodological correction on antenna closure that we had previously got wrong in print (Section IX).

II. RELATED WORK

Open-PDK silicon for machine learning has concentrated on inference of discriminative models, where the activation functions are typically ReLU and therefore free. Generative networks need bounded, curved nonlinearities — tanh at the generator output, sigmoid at the discriminator output — and the usual on-chip answers are lookup tables, CORDIC iterations, or piecewise-linear approximation. Table lookup is the natural choice in simulation and the worst choice in silicon: the 8193-entry table in our own prior work would have dominated the die. Piecewise-linear approximation of sigmoidal functions is long established; what we add is a measurement of the end-to-end cost, in output pixels rather than in function-domain error, of moving a generator from an exact table to a nine-segment fit, and the observation that this is the change that makes the generator implementable at all.

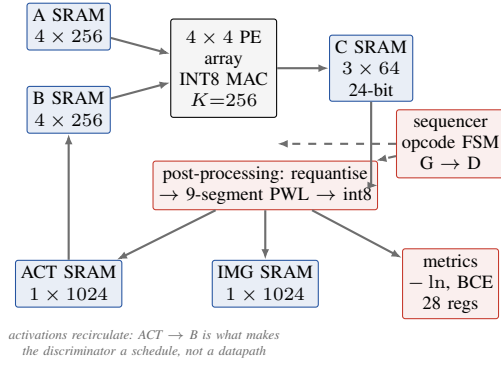


Fig. 1. The engine. Solid arrows are data, dashed are control. The discriminator adds no block to this figure: it reuses the same array, the same buffers and the same post-processing, differing only in where the sequencer sends results.

III. ARCHITECTURE

Fig. 1 shows the engine. The unchanged INT8 MAC engine from the prior design sits inside a new top level that adds a sequencer, a post-processing pipeline, a metrics block and two deep buffers.

A. The discriminator is a schedule, not a datapath

The natural assumption is that adding a discriminator to a generator accelerator roughly doubles it. It does not. Both networks are dense INT8 layers followed by a bounded activation; they differ in shape, not in kind. In the RTL the entire distinction between a generator layer and a discriminator layer is two values of a destination-select configuration field plus one extra opcode that latches a loss. The MAC array, the operand buffers, the requantiser, the activation unit and the drain paths are shared without modification.

The practical consequence, which we quantify in Section V and Section VII, is that removing the discriminator to save area would save approximately nothing while costing the forward half of every training step. The same accounting applies at the pins: of the pads this chip uses beyond its predecessor, exactly one is attributable to the discriminator, and even that one is a convenience — it exposes the verdict bit to an oscilloscope, and the same bit is readable over the serial link.

B. Sequencer

A single opcode FSM drives the whole adversarial pass: load a weight tile, run the array, flush results through post-processing to a destination, recirculate activations from the activation buffer back into the B operand buffer, score, and latch losses. The host issues opcodes over the serial link; no microcode memory is needed because the opcode set is small and the layer schedule is fixed by configuration registers rather than by instruction sequences.

IV. PIECEWISE-LINEAR ACTIVATIONS

All six functions the network needs — tanh, sigmoid, ReLU, LeakyReLU(0.2), identity, and a $\log_2$ mantissa used by the loss unit — share one datapath and one multiplier, selected by a

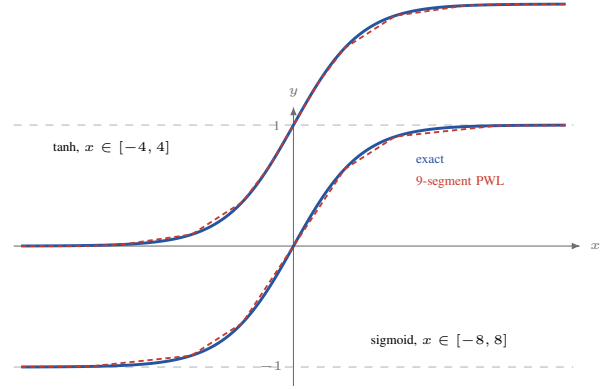


Fig. 2. Nine-segment approximation against the exact functions, evaluated through the same integer datapath the RTL uses. Curves are drawn on independent horizontal scales; sigmoid is shown over twice the domain of tanh.

configuration field. Each is a nine-segment piecewise-linear ladder in Q4.12: a comparator chain selects the segment, and one multiply-add evaluates $y = m_i x + c_i$.

Fig. 2 plots the result against the exact functions. Maximum absolute error is 0.04133 for tanh and 0.01966 for sigmoid; the log mantissa fit, which is minimax-fitted rather than inherited, is accurate to 5.16 LSB, and the resulting $-\ln$ used for the cross-entropy losses is accurate to about 0.001 nats.

Function-domain error is the wrong metric for a generator, because what matters is the pixel that comes out. Rendering a complete MNIST digit through the approximation and through the 8193-entry table it replaces gives a mean absolute difference of **1.71 gray levels out of 255**. That is the entire accuracy cost of making the generator implementable.

Two deliberate departures from the reference implementation we ported the coefficients from are worth recording. First, one shared multiplier rather than one per function: the functions are mutually exclusive in time, so the area of five extra multipliers buys nothing. Second, the output is clamped to each function’s mathematical range. The reference’s outermost sigmoid segments have non-zero slope, so far from the origin it returns values outside $[0, 1]$ — a negative “probability” that would corrupt the cross-entropy loss downstream.

A. Folded gain, and why it is exact

Pre-activations in a Q4.12 word saturate at ± 8 , while the real pre-activations of the ReLU layers reach ± 180 . Rather than widen the word, a per-layer gain is folded into the requantisation multipliers and undone after the activation. This is exact, not approximate, for the ReLU family because those functions are positively homogeneous: $f(gx) = g f(x)$. The tanh and sigmoid layers are not homogeneous and are therefore forced to unit gain — which costs nothing, because both functions are already flat where the saturation would bite.

V. MEMORY RIGHT-SIZING AND BATCHING

The available SRAM family has a fixed width of $301.3 \mu\text{m}$ and scales only in height, so area per stored byte improves

TABLE I
OPERAND MEMORY, RIGHT-SIZED ACROSS THREE DEPTHS

bank	macros	bytes	area (μm^2)
A (weights)	$4 \times 256 \times 8$	1024	271,086
B (inputs)	$4 \times 256 \times 8$	1024	271,086
C (accumulators)	$3 \times 64 \times 8$	192	137,583
ACT (activations)	$1 \times 1024 \times 8$	1024	155,414
IMG (image)	$1 \times 1024 \times 8$	1024	155,414
total	13 macros	4288	990,581
prior uniform	$16 \times 256 \times 8$	4096	1,084,343

TABLE II
MEASURED HOST-LINK COST FOR ONE 1024-BYTE WEIGHT TILE

mode	SCLK edges	per byte	speed-up
single-byte frames	24,576	24.00	1.00×
serial burst	8,208	8.01	2.99×
parallel burst	1,040	1.01	23.63×

steeply with depth: 717, 265, 189 and $152 \mu\text{m}^2$ per byte at depths 64, 256, 512 and 1024 respectively. A uniform choice of macro is therefore a poor one. Table I shows the arrangement adopted.

The result is 8.6% less macro area than the uniform arrangement, with 91% bank utilisation, while simultaneously enabling four-way batching. A and B need four parallel byte lanes and cannot be merged — A holds four weight rows, B four batch columns — but C never holds more than $N^2 = 16$ words, so 256-deep macros there wasted 94% of their depth, and the two new buffers want depth rather than width.

Batching follows from the array’s shape. The 4×4 array was always a matrix–matrix engine: B’s four columns can hold four independent input vectors, and C’s sixteen words are then four neurons across four lanes. One streamed weight tile therefore serves four images. Because weight streaming dominates run time — a matrix–vector product has no weight reuse whatsoever, at 1.01 weight bytes per MAC — this is close to a linear win: **103,398 compute cycles per digit at batch 4 against 440,252 at batch 1, a $4.26\times$ improvement**, verified end to end with all four lanes bit-exact. Four full images do not fit on chip, so the host holds them and the image buffer becomes a drain window; the accuracy cost of sharing one calibration across four latents is 0.15–0.59 gray levels.

VI. HOST LINK

The chip talks to its host over four wires. In the base protocol every byte costs a 24-edge frame, which makes the link, not the array, the entire latency of the system. Two burst commands remove that. `WR_BURST` sends the start address once and auto-increments, costing 8 edges per byte; `WR_BURST8` additionally takes the byte off an 8-bit parallel bus, costing one edge per byte. Measured at the pads over a full 1024-byte tile:

Counting every host write, including the discriminator’s 784-wide input which must be re-fed per weight tile at batch 4,

TABLE III
SIGN-OFF OF THE HARDENED ENGINE MACRO

metric	value
die / macros	$1900 \times 1900 \mu\text{m}$, 13 SRAM
clock	40 ns (25 MHz), 9 corners
setup / hold worst slack	+12.30 ns / +0.118 ns
Magic DRC / LVS / XOR	0 / 0 / 0
routing DRC	0
antenna nets / pins	0 / 0
power (typical)	228 mW
reproducibility	byte-identical placement DB

the two mechanisms together take end-to-end latency from **9.45 s to 0.161 s per image**, a $58.5\times$ improvement. The $4.26\times$ compute figure is separate from and unaffected by this.

The parallel bus is only worth its eight pads if the host can write a byte in one register store, which requires allocating those pads to eight contiguous bits of one GPIO port. Scattered across ports, the host spends eight bit-bang writes per byte and the advantage is gone.

VII. PINS

The engine’s internal interface is 106 signal bits, and the target shuttle slot offers 20 general-purpose digital pads. Serialisation absorbs the difference: 17 pads are used — four for the link, four for status, eight for the parallel burst bus, and one mirroring the readback line so that a single failed bond cannot blind the chip — and three remain spare. Minimum viable bring-up needs seven signals plus one supply rail: clock, reset, the four link wires, and a busy flag. Opcodes are not pins; they travel inside the address field of the link frame, which is why the command set can grow without touching the pad budget.

VIII. HOST-IN-THE-LOOP TRAINING

Because every activation on this chip has a derivative expressible in terms of its own *output* — $\text{ReLU}' = [a > 0]$, $\tanh' = 1 - a^2$, $\sigma' = a(1 - a)$ — and because those outputs already drain over the existing read commands, host-side backpropagation needs no new hardware. The chip performs the forward pass of both networks; the host keeps float master weights, recalibrates per step, and applies the optimiser. A driver closes this loop and is bit-exact against the golden model. The cost is roughly 20% additional link traffic for the drains, which a burst *read* command would largely remove — the one addition to this design we would recommend making before silicon.

IX. PHYSICAL IMPLEMENTATION

The engine hardens as a single macro on a $1900 \times 1900 \mu\text{m}$ die with all 13 SRAM macros placed, using the same native-3.3 V standard-cell library and single-supply operating point as the prior chip. Table III reports the sign-off.

The clock was set by measurement rather than estimate. The first routed run closed a deliberately conservative 60 ns target with +27.56 ns of slack, i.e. a real critical path of 32.44 ns, which showed that the 28×20 array multiplier added by

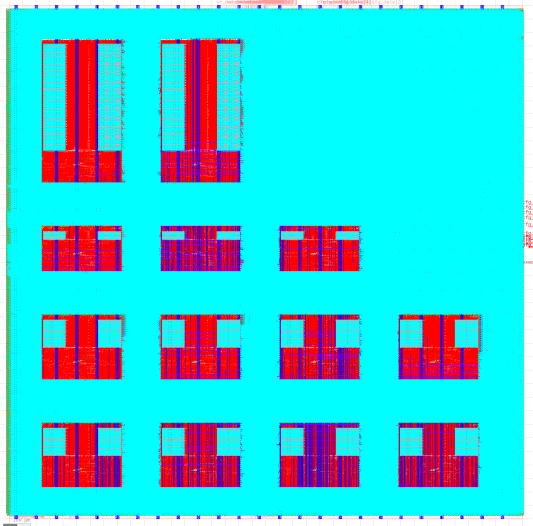


Fig. 3. The hardened engine macro, $1900 \times 1900 \mu\text{m}$. The right-sized memory is visible as three distinct macro heights: the two 1024-deep buffers at the top, the three 64-deep accumulator byte-planes in the middle, and the eight 256-deep operand macros below.

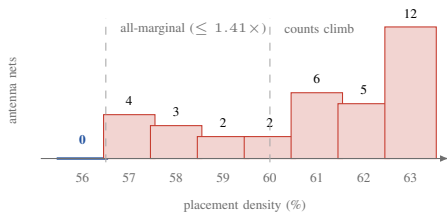


Fig. 4. The full sweep. The earlier conclusion was drawn from the samples at 58 and above; the value that closes is below all of them.

post-processing was not the bottleneck it had been sized for. Retiming to 40 ns matched the prior chip exactly and scaled power by precisely $1.5\times$, identifying the macro as dynamic-power dominated.

A. A correction on antenna closure

Antenna violations on this process are a per-net, per-layer metal-area rule, non-gating in this flow but worth closing as a quality goal. Our prior work closed them by perturbing the placement density, and we reported that lever as behaving like a re-roll rather than an optimisation.

On this design the same lever initially appeared exhausted. Sampling densities 58 through 63 gave counts of 3, 2, 2, 6, 5 and 12, a floor of two reached from both sides, and the survivors were not marginal: each was a $> 2.7\times$ violation on a long broadcast net between an SRAM macro and the array. We concluded in an earlier draft of this work that the cause was structural and that only a floorplan change could fix it.

That conclusion was wrong, and Fig. 4 shows why. Every sample had been taken at or above the starting point of 60. Extending the sweep *downward* not only reduces the count but changes the kind of violation: structural at 59–60, all-marginal at 57–58, and none at all at 56.

TABLE IV
THIS WORK AGAINST THE PRIOR ACCELERATOR

	prior [1]	this work
on chip	generator MACs	G + D, losses, metrics
activations	testbench LUT	9-segment PWL, on die
SRAM macros	11 ($\times 256$)	13 (64/256/1024)
buffer bytes	2816	4288
batch lanes	1	4
engine die	$1600 \times 1500 \mu\text{m}$	$1900 \times 1900 \mu\text{m}$
clock	40 ns	40 ns
antenna	0 / 0	0 / 0
digital pads	7 of 20	17 of 20
end-to-end	1.61 s/image	0.161 s/image

The mechanism in the original reading was right — the $> 2.7\times$ nets really are long macro-to-array broadcasts, and they really do not move anywhere between 59 and 63. What was wrong was the inference from “this lever cannot shift these nets” to “no value of this lever can”. A looser placement leaves the router room to detour a long net; a tighter one takes it away, which is why the counts climb steeply above 60. We state the correction explicitly because the failure mode is easy to repeat: a lever sampled on only one side of its starting point can look exhausted when it has not been explored. The result is reproducible in the strong sense — re-running the promoted configuration produces a byte-identical placement database.

Should the problem recur after a netlist change, the evidence points at a more targeted lever than either density or floorplan: every violation across all ten routed runs of this design was on the same metal layer, so reducing that layer’s routing capacity should push the long nets upward, where the upper layer is nearly empty.

B. What changed relative to the prior chip

Table IV summarises the difference. The engine grew from a matrix-multiply accelerator into a complete adversarial pipeline, and the die grew with it, but the external interface did not: the same shuttle slot, the same two dedicated pads, the same single supply, and three spare general-purpose pads left over.

C. Chip-level integration

The macro is then placed inside the shuttle padding, behind the serial bridge, and the complete chip is taken through the same flow. One template setting had to change. The padding’s own configuration enables the detailed router’s in-place antenna repair, which on this chip aborts routing with a no-access-point error on a diode the repair itself had just placed — the same wall this project documented from five directions on the engine macro, where any non-zero repair iteration count triggers a verification re-route whose pin-access phase must reach diodes that have none. The prior, smaller chip survived the same setting; this one, with roughly twice the glue logic and more pre-route violations, does not. Disabling that repair lets routing complete.

X. VERIFICATION

Correctness is checked against a single Python golden model at every level. The activation and loss units match over 1858 directed vectors. A full adversarial pass matches bit-exactly on all 784 pixels plus both scores, both losses and every metric register. The four-way batched flow matches on **3136 of 3136 pixels** across four lanes simultaneously, with the four scores distinct and the replicated real-image scores identical — a lane-crossing bug would break that symmetry. The pad-level testbench drives the physical pins rather than internal ports and reproduces the measured burst edge counts of Table II. The prior chip’s own regression suite passes unchanged.

XI. DISCUSSION AND LIMITATIONS

Three limitations are worth stating plainly. The loss registers saturate at 8.32 nats, because a Q4.12 probability cannot represent $y < 2^{-12}$; this affects only the reported metric, not the computation. The activation approximation costs 1.71 gray levels, which is invisible in a rendered digit but is not zero. And the shared calibration across a batch costs a further 0.15–0.59 gray levels, which is the price of the $4.26\times$.

The broader observation from this work is about where the boundary between hardware and software actually falls in a small generative model. The discriminator, which sounds like a second accelerator, is a schedule. The training loop, which sounds like it needs dedicated gradient hardware, needs none, because every activation’s derivative is a function of its own output and those outputs already leave the chip. What genuinely had to become hardware was the least glamorous part: a nine-segment approximation to two curves.

XII. CONCLUSION

A complete GAN — generator, discriminator, bounded activations, both cross-entropy losses and their metrics — fits on an open-PDK 180 nm die behind an unchanged pin budget, and hardens with zero DRC, LVS and antenna violations at 40 ns on a single 3.3 V supply. Right-sizing the operand memory across three depths made the design 8.6% smaller while paying for $4.26\times$ batching, and burst commands on the host link took end-to-end latency from 9.45 s to 0.161 s per image. The parts that were hard were not the parts that sound hard.

ACKNOWLEDGMENT

The authors thank the open-source EDA community, the GlobalFoundries open PDK program for GF180MCU and its SRAM macros, and the wafer.space chipathon program for the padding template and shuttle slot targeted by this work.

REFERENCES

- [1] W. Anthony, I M. M. Surya W.M., J. R. Saragi, M. P. Adipratama, and F. A. Rahagi, “An 8-bit deep learning accelerator for GAN image generation on GF180MCU,” in *Proc. APSIPA ASC*, 2026.
- [2] GlobalFoundries and Google, “GF180MCU open process design kit,” 2022. [Online]. Available: <https://github.com/google/gf180mcu-pdk>
- [3] LibreLane Contributors, “LibreLane: an open-source RTL-to-GDSII flow,” 2025. [Online]. Available: <https://github.com/librelane/librelane>

- [4] T. Ajayi *et al.*, “OpenROAD: toward a self-driving, open-source digital layout implementation tool chain,” in *Proc. GOMACTech*, 2019.
- [5] I. Goodfellow *et al.*, “Generative adversarial nets,” in *Adv. Neural Inf. Process. Syst.*, 2014.
- [6] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE CVPR*, 2018.